

Architecture Primitives for Private Image Classification under Fully Homomorphic Encryption

Yavor Litchev
Stanford University
ylitchev@stanford.edu

Abstract

Fully homomorphic encryption (FHE) permits model evaluation on encrypted images without revealing the input to the server, but the computational cost of encrypted neural-network inference remains a major obstacle. This work examines a practical computer-vision question: for small CNN classifiers, which architectural primitives preserve accuracy and latency characteristics during the transition from plaintext inference to quantized FHE inference? Rather than treating FHE solely as a deployment backend, the study treats it as an architectural constraint. Compact MNIST and FashionMNIST classifiers are trained in plaintext, compiled with Concrete ML using exact post-training quantization, and evaluated under plaintext, quantized-cleartext, simulated-FHE, and selected true-FHE CUDA execution. The primary empirical finding is that operator structure matters more than raw parameter count. On MNIST, a square-activation strided CNN reaches 92.34% full-test accuracy, preserves 96.88% accuracy on a 32-image FHE probe, and completes true CUDA FHE inference in 579.7 seconds on one encrypted sample. A compact strided variant executes faster at 569.0 seconds, with lower full-test accuracy. On FashionMNIST, the same primitive family remains feasible but exposes a larger accuracy-quantization gap. These results indicate that FHE-aware architecture design is a distinct model-design problem, not merely a matter of pruning or selecting compiler options.

1. Introduction

Computer vision systems are increasingly deployed as services: a user sends an image to a remote model, and the server returns a prediction. Although operationally convenient, this deployment model is problematic when the image contains sensitive information

such as medical scans, identity documents, financial records, or private user behavior. Fully homomorphic encryption (FHE) offers an appealing alternative: the client encrypts the image, the server evaluates a model directly on ciphertexts, and the client decrypts the returned prediction. In the ideal protocol, the server never observes the image in plaintext.

The difficulty is that neural-network inference under FHE is not equivalent to plaintext inference executed on a slower backend. Encrypted execution changes which operations are relatively inexpensive, which operations are expensive, and which quantized circuits are feasible. A small CNN that is efficient on a GPU may compile into an expensive encrypted circuit, while a slightly larger network with different primitives may execute faster. This work addresses the following focused question:

Which simple CNN architectural primitives best preserve the accuracy-latency tradeoff when a plaintext image classifier is compiled for FHE inference?

The input to the vision model is a single 14×14 grayscale image from MNIST or FashionMNIST, scaled to $[0, 1]$. A compact CNN outputs logits over ten classes; after Concrete ML compilation, the encrypted protocol returns the same ten-class prediction while preventing the server from observing the input image in plaintext.

The initial implementation evaluated LeNet-style baselines, pruning, distillation, and quantization-aware training (QAT). Those experiments served as diagnostic baselines and revealed that parameter reduction alone is insufficient: a 401-parameter pruned model executed more slowly than a 562-parameter reference under true CUDA FHE. The project was therefore refined into a controlled primitive-ablation study that varies activation, downsampling, and classifier head while fixing dataset, training budget, quantization, calibration size, and benchmark protocol.

The contributions are:

- A reproducible PyTorch/Concrete ML pipeline for plaintext training, exact post-training quantization (PTQ), simulated FHE, and selected true-FHE execution.
- A controlled ablation of CNN primitives: ReLU, square, and identity activations; average-pooling and strided-convolution downsampling; dense, compact, and global-average-pooling heads.
- Empirical evidence that strided downsampling improves the encrypted-inference frontier on MNIST and transfers partially to FashionMNIST at 14×14 resolution.
- Negative results showing that pruning, ordinary distillation, basic QAT, and removing nonlinearities do not automatically improve true-FHE run-time.

2. Related Work

Encrypted neural-network inference. CryptoNets demonstrated neural-network inference over encrypted data and emphasized the need to adapt neural architectures to the arithmetic supported by homomorphic encryption [6]. Follow-up work such as Faster CryptoNets studied sparsity, quantization, and activation approximations to improve encrypted inference [3]. LoLa showed that latency can be improved by carefully choosing representations and linear algebra strategies [1]. CHET approached the problem from the compiler side, automatically selecting tensor layouts and encryption parameters for homomorphic tensor programs [4]. These works motivate the central premise of this study: the architecture and operator representation are part of the FHE system design.

FHE systems and tooling. FHE was first shown to be possible by Gentry [5]. Modern schemes and implementations, including TFHE-style programmable bootstrapping [2], have made encrypted computation more practical. This study uses Concrete ML, an open-source framework that compiles scikit-learn and PyTorch-style models into FHE equivalents [12]. Concrete ML is well suited for this work because it provides a practical workflow while exposing latency and compilation failures.

Quantization and small vision models. Low-bit quantization is a standard technique for efficient inference [7], and it is also necessary for Concrete ML compilation. However, FHE changes the objective: the relevant question is not only whether quantization preserves plaintext accuracy, but whether the resulting

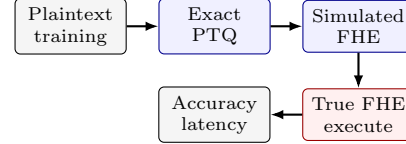


Figure 1. Pipeline used in the study. Models are trained in plaintext, then compiled and evaluated under quantized and encrypted inference modes.

quantized circuit remains feasible under encrypted execution. MNIST and LeNet-style models provide a controlled vision setting [8], while FashionMNIST provides a more challenging cross-dataset transfer check [10]. Models are implemented in PyTorch [9]. QAT experiments use Brevitas-style quantized PyTorch modules [11].

3. Encrypted Inference Setup

3.1. Threat model and execution modes

The evaluation considers encrypted *inference*, not encrypted training. Let x be a client image, f_θ a trained classifier, Enc_{pk} encryption with a public key, and Dec_{sk} decryption with the client secret key. The target protocol is

$$\hat{y} = \text{Dec}_{sk} \left(\tilde{f}_\theta(\text{Enc}_{pk}(x)) \right), \quad (1)$$

where $\tilde{f}_\theta$ is the compiled FHE circuit. The server evaluates $\tilde{f}_\theta$ but does not see x or the decrypted prediction.

Concrete ML exposes three evaluation modes. **disable** runs the quantized model without encryption and measures quantization effects. **simulate** simulates the FHE computation without real encryption and is efficient enough for broad sweeps. **execute** performs true FHE evaluation on encrypted inputs and is reserved for selected candidates because each sample can take minutes.

3.2. Quantization

For PTQ models, Concrete ML maps real-valued activations and weights into low-bit integer representations. A simplified affine quantizer is

$$q(x; s, z, b) = \text{clip} \left(\left\lfloor \frac{x}{s} \right\rfloor + z, 0, 2^b - 1 \right), \quad (2)$$

where s is a scale, z is a zero point, and b is the bit-width. The experiments primarily use exact 4-bit and 5-bit PTQ with 512 calibration images. Exact rounding was necessary because earlier approximate rounding settings produced incorrect CNN predictions.

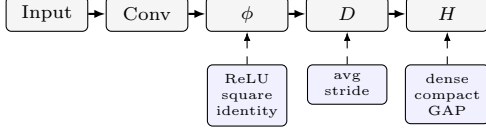


Figure 2. Primitive CNN design. The experiment isolates activation, downsampling, and classifier-head choices while holding the dataset and FHE compilation protocol fixed.

4. Methods

4.1. Primitive CNN family

The main model family is a compact configurable CNN. Given image x , convolution C_i , activation ϕ , downsampling operator D , and classifier head H , the model is

$$h_1 = D(\phi(C_1(x))), \quad (3)$$

$$h_2 = D(\phi(C_2(h_1))), \quad (4)$$

$$\hat{y} = H(h_2). \quad (5)$$

The ablation varies the following primitives:

- $\phi \in \{\text{ReLU}, x^2, \text{identity}\}$.
- $D \in \{\text{AvgPool}, \text{strided convolution}\}$.
- $H \in \{\text{dense}, \text{compact}, \text{GAP}\}$.

The dense head has two hidden layers, the compact head removes one hidden layer, and the GAP head applies global average pooling followed by a linear classifier.

4.2. Compression baselines

Prior to the primitive study, three common compression approaches were evaluated. Structured pruning extracts narrower LeNet-style subnetworks and fine-tunes them. Distillation trains a student from a teacher using

$$\mathcal{L}_{\text{distill}} = (1 - \alpha)\text{CE}(y, p_s) + \alpha T^2 \text{KL}(p_t^{(T)} \| p_s^{(T)}), \quad (6)$$

where T is temperature and α is the soft-target weight. QAT trains Brevitas quantized models before Concrete ML compilation. These baselines evaluate the hypothesis that conventional model compression is sufficient for FHE. The experiments below show that this hypothesis is not supported.

5. Experiments

5.1. Dataset and training protocol

The primary experiments use MNIST resized to 14×14 grayscale images. MNIST contains 60,000 training images and 10,000 test images, each originally represented as a 28×28 grayscale digit image. This setting

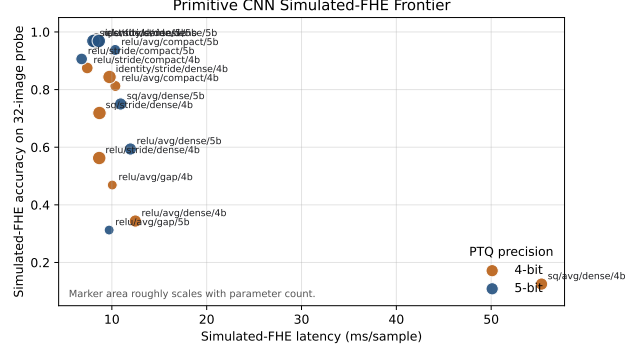


Figure 3. Simulated-FHE frontier for primitive CNNs. Marker area roughly reflects parameter count. Several 4-bit variants have low latency but low accuracy; 5-bit strided variants preserve accuracy more reliably.

is deliberately compact: true FHE execution is expensive, and the goal is to study architectural effects in a controlled setting rather than maximize benchmark accuracy. To test whether the conclusion is specific to MNIST, a targeted FashionMNIST transfer study uses the same resolution, training budget, quantization settings, calibration size, and benchmark size. FashionMNIST also contains 60,000 training images and 10,000 test images, but its clothing categories are visually more ambiguous than digit recognition and therefore test whether the primitive choices still provide sufficient capacity after aggressive resizing. Images are resized, converted to tensors scaled to $[0, 1]$, and not augmented.

Models are trained in plaintext for 8 epochs with Adam, learning rate 0.003, batch size 256, and seed 231. FHE compilation uses exact PTQ, 512 calibration images, and a 32-image held-out probe for `disable` and `simulate`. Selected models are then run in true CUDA FHE on one encrypted sample. The primary classification metric is accuracy,

$$\text{Acc} = \frac{1}{n} \sum_{i=1}^n \mathbf{1} \left[\arg \max_k f_{\theta}(x_i)_k = y_i \right], \quad (7)$$

reported on the full test set and on the 32-image FHE probe. The systems metrics are simulated latency, compile time, key-generation time when available, and true-FHE wall-clock execution time.

5.2. Primitive ablation results

Figure 3 plots simulated-FHE accuracy against simulated latency. Table 1 reports the exact 5-bit rows, which are the most informative because many 4-bit variants suffer severe simulated-accuracy degradation.

The dominant pattern is that strided downsampling dominates average pooling. ReLU+stride+dense,

Activation	Head	Params	Test	Sim.	Sim. ms
ReLU avg	dense	1074	.7149	.5938	11.956
Square avg	dense	1074	.9411	.7500	10.923
ReLU avg	GAP	306	.7268	.3125	9.713
ReLU avg	compact	642	.9266	.9375	10.347
ReLU stride	dense	1522	.9282	.9688	8.370
ReLU stride	compact	866	.8992	.9062	6.814
Square stride	dense	1522	.9234	.9688	8.049
Identity stride	dense	1522	.9085	.9688	8.627

Table 1. Primitive ablation at exact 5-bit PTQ. “Test” is full MNIST test accuracy. “Sim.” is accuracy on the 32-image simulated-FHE probe.

Variant	4-bit sim.	5-bit sim.	Change
ReLU avg dense	.3438	.5938	+.2500
Square avg dense	.1250	.7500	+.6250
ReLU stride dense	.5625	.9688	+.4063
ReLU avg compact	.8125	.9375	+.1250
Square stride dense	.7188	.9688	+.2500
Identity stride dense	.8438	.9688	+.1250

Table 2. Representative simulated-FHE accuracy changes when moving from exact 4-bit to exact 5-bit PTQ on the 32-image probe.

square+stride+dense, and identity+stride+dense all preserve 96.88% probe accuracy at 5 bits. By contrast, the original ReLU+average-pooling dense model reaches only 59.38% simulated accuracy. The compact head yields the lowest simulated latency, but sacrifices full-test accuracy. The GAP model confirms that parameter count alone is an insufficient design objective: it has only 306 parameters but is not competitive.

5.3. Ablation-axis analysis

Bit width. The 4-bit versus 5-bit comparison is one of the clearest findings. A lower bit-width is attractive because it should reduce encrypted arithmetic cost, but it often substantially degrades classifier accuracy before true FHE execution is appropriate to measure. Table 2 shows representative transitions. The ReLU+stride+dense model improves from 56.25% to 96.88% simulated-FHE probe accuracy when moving from 4 to 5 bits. The square+average-pooling model improves from 12.50% to 75.00%, but still remains far below its plaintext behavior. This means that bit-width should be treated as a correctness constraint. For the final true-FHE measurements, the primitive variants therefore use 5-bit PTQ even though 4-bit circuits are sometimes less expensive.

Downsampling. Strided convolution is the most robust primitive in the study. This is not simply because it has fewer operations: the strided dense models have 1522 parameters, more than the average-pooling dense reference. One possible explanation is that the learned downsampling operator controls the feature distribution before later quantized layers, whereas fixed average pooling can pass activation ranges that are

more difficult for a low-bit compiler to represent accurately. Although this mechanism is not established causally here, the empirical pattern is consistent across ReLU, square, and identity activations.

Activation. The square activation is historically motivated by CryptoNets-style arithmetic circuits, but the results indicate that it is not universally superior. With average pooling, square activation trains well in plaintext but degrades substantially after quantization. With strided convolution, square activation becomes one of the strongest variants, matching the ReLU strided model on simulated accuracy and slightly improving true-FHE latency. This interaction is important: evaluating an activation in isolation can be misleading because its quantized behavior depends on the rest of the circuit.

Classifier head. The classifier-head ablation provides an accuracy-latency frontier. Global average pooling is overly restrictive: it minimizes parameters but loses too much capacity. The compact head is more informative. ReLU+stride+compact has only 866 parameters and is the fastest simulated and true-FHE primitive candidate, but its full-test accuracy is 89.92%, below the dense strided variants. This provides a controllable operating point for applications where latency is more important than full-test accuracy.

5.4. Implementation and reproducibility

The project code stores each experiment as a YAML configuration so that architectural choices are part of the artifact metadata. Training artifacts are saved as PyTorch checkpoints and JSON histories. Compilation artifacts are JSON summaries containing quantization bit-width, calibration size, compile time, plaintext accuracy, simulated-FHE accuracy, and true-FHE latency when available. The final plots are generated from CSV tables rather than hand-entered values. This is important because true-FHE experiments are slow and easy to misreport; preserving the JSON output from each execute run makes the paper tables auditable.

5.5. True FHE execution

True FHE execution is the most important but most expensive measurement. Table 3 and Figure 4 compare selected one-sample CUDA `execute` runs. All listed execute rows made the correct prediction on the encrypted sample.

Among the high-accuracy true-FHE candidates, square+stride+dense performs best: it has 92.34% full-test accuracy and executes in 9.66 minutes. The lowest-latency true-FHE candidate is ReLU+stride+compact

Model	Params	Test	Bits	Execute min
Micro-LeNet PTQ	562	.8419	4	16.06
Micro-LeNet QAT	569	.7699	4	18.56
ReLU stride dense	1522	.9282	5	10.00
Square stride dense	1522	.9234	5	9.66
Identity stride dense	1522	.9085	5	10.36
ReLU stride compact	866	.8992	5	9.48

Table 3. Actual CUDA FHE execution on one encrypted test image. Parameter count does not explain runtime: the 1522-parameter square-stride model is faster than the 562-parameter PTQ micro-LeNet.

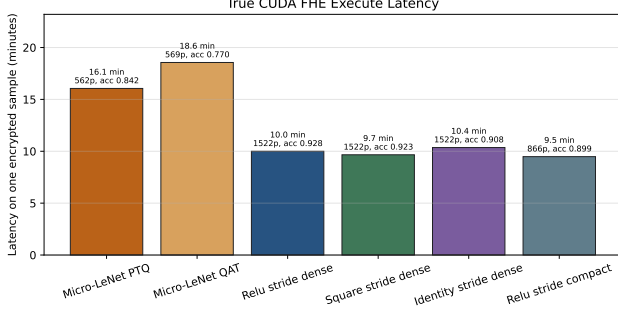


Figure 4. True CUDA FHE execute latency. The primitive strided models improve the runtime frontier relative to earlier PTQ and QAT micro-LeNet baselines.

at 9.48 minutes, with full-test accuracy of 89.92%. The identity model is slower than square and ReLU dense models despite removing nonlinear activations, so the simple heuristic “remove nonlinearities” is not sufficient in this Concrete ML setting.

5.6. FashionMNIST transfer study

MNIST provides a controlled setting for systems experiments, but it is visually less complex than many practical image-classification tasks. To test whether the primitive findings survive a more challenging image classification task, a targeted subset of the primitive sweep was repeated on FashionMNIST. The full MNIST grid was not repeated; instead, the selected variants include the reference average-pooling model, the ReLU strided model, the square strided model, and the compact strided model. These variants address the principal transfer questions: whether average pooling continues to underperform, whether strided downsampling still helps, whether square activation remains competitive, and whether compact heads remain a latency-favoring option.

The FashionMNIST results support the analysis in two ways. First, the downsampling conclusion transfers: every strided model has much higher full-test accuracy than the average-pooling reference, and the strided models are faster in simulated FHE. Second, the more challenging dataset exposes a limitation that

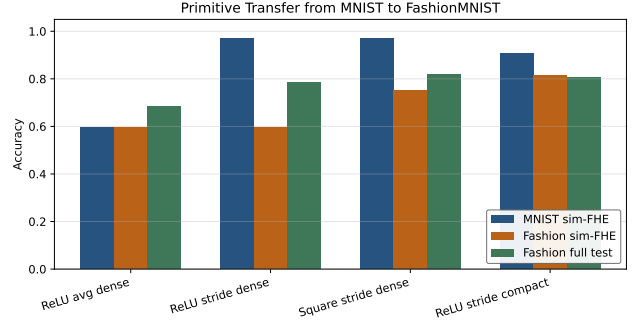


Figure 5. Primitive transfer from MNIST to FashionMNIST at exact 5-bit PTQ. FashionMNIST is more challenging, but the strided variants still dominate the average-pooling reference.

FashionMNIST variant	Params	Test	Sim.	Sim. ms
ReLU avg dense	1074	.6829	.5938	11.540
ReLU stride dense	1522	.7842	.5938	8.222
Square stride dense	1522	.8179	.7500	8.086
ReLU stride compact	866	.8056	.8125	7.211

Table 4. FashionMNIST exact 5-bit primitive-transfer results. “Test” is full FashionMNIST test accuracy, while “Sim.” is the 32-image simulated-FHE probe accuracy.

is less visible on MNIST. The full-test and probe accuracies no longer align as cleanly. For example, square+stride+dense has the best full-test accuracy at 81.79%, but ReLU+stride+compact has the best simulated-FHE probe accuracy at 81.25%. This suggests that for more challenging data, the final architecture choice may depend on whether the deployment priority is full-test accuracy, quantized robustness on the target distribution, or encrypted latency.

True CUDA FHE execution was also evaluated for the FashionMNIST ReLU+stride+compact model. It completed with a correct prediction on one encrypted sample in 567.2 seconds, with 2.76 seconds for key generation. This runtime is close to the MNIST compact-stride runtime, as expected because the architecture and input shape are the same. This run is therefore intended not as a new systems speedup result, but as an end-to-end feasibility check showing that the FashionMNIST transfer candidate also compiles and executes under actual FHE.

5.7. FashionMNIST failure modes

Figure 6 shows the full-test plaintext confusion matrix for the FashionMNIST square+stride+dense model. This is not an encrypted-execution measurement; it is included to explain why FashionMNIST is a more demanding transfer test. The most visible confusions are among visually related clothing categories such as shirts, T-shirts, pullovers, and coats. This mat-

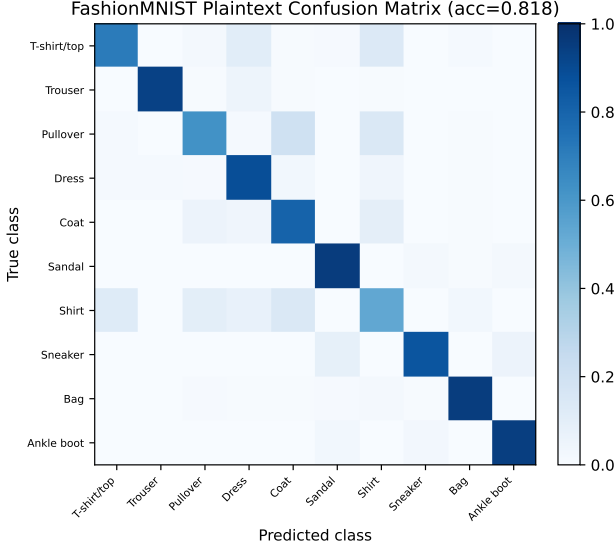


Figure 6. Row-normalized plaintext confusion matrix for the FashionMNIST square+stride+dense model. The main errors are concentrated among similar upper-body clothing categories, explaining why FashionMNIST exposes a larger robustness gap than MNIST.

ters for the FHE study because the privacy-preserving classifier is already forced into a small 14×14 , low-bit regime. If the plaintext model is uncertain among semantically similar categories, quantization has less margin before the decision boundary changes.

This failure analysis also clarifies how the FashionMNIST results should be interpreted. The square+stride model is the best full-test classifier, while the compact strided model has the best 32-sample simulated-FHE probe. That disagreement is not necessarily a contradiction; it indicates that on more challenging data, a small FHE probe may emphasize different examples than the full test set. A final deployment would require a larger encrypted or simulated-FHE validation slice, but the current result is already informative for the analysis: it shows that the primitive trend transfers, while also revealing that more challenging visual categories require more careful validation.

5.8. Compression and QAT results

Table 5 summarizes the compression path. It primarily served as a negative result. Pruning reduced parameters but did not reduce true-FHE latency. Distillation improved plaintext accuracy for a pruned student but degraded exact quantized behavior. QAT produced models that preserved quantized behavior more consistently, but the best QAT execute run was still slower than the tuned PTQ reference.

Model	Params	Test	Exec. min	Outcome
Micro-LeNet PTQ	562	.8419	16.06	reference
Pruned nano	401	.8021	17.18	higher latency
Pruned nano 5b	401	.8021	22.12	higher latency
Distilled nano	401	.8225	—	lower sim. acc.
Micro QAT float	569	.7699	18.56	higher latency
Square stride dense	1522	.9234	9.66	best accuracy

Table 5. Compression baselines versus the strongest primitive result. Conventional compression did not automatically improve encrypted runtime.

6. Discussion

The central conclusion is that FHE-aware model design is not equivalent to ordinary model compression. Parameter count, plaintext latency, and even simulated latency are imperfect proxies for true encrypted execution. The primitive study suggests three practical rules.

First, downsampling is a high-leverage primitive. Replacing average pooling with strided convolution consistently moved models toward higher simulated-FHE accuracy and lower latency. Second, bit width is a correctness constraint, not only a speed knob. Several 4-bit variants were less expensive but unsuitable because quantization substantially degraded accuracy. Third, classifier-head compression creates a real accuracy-latency tradeoff: the compact strided model is fastest in true FHE but sacrifices full-test accuracy.

There are also limitations. The experiments use MNIST and FashionMNIST at 14×14 resolution, so the absolute accuracies and latencies should not be interpreted as state-of-the-art vision performance. The FashionMNIST transfer study is targeted rather than exhaustive, and the 32-image FHE probe is too limited to estimate final deployment accuracy. True-FHE measurements use one encrypted sample because each execution takes approximately ten minutes. The security parameters are delegated to Concrete ML rather than independently audited. Finally, the architecture search is intentionally compact; it is a controlled ablation, not an exhaustive neural architecture search.

7. Conclusion

This work studied private image classification under FHE as a computer-vision architecture problem. The empirical evidence supports a concrete thesis: simple CNN primitive choices can move the encrypted accuracy-latency frontier more effectively than parameter reduction alone. Strided-convolution downsampling was the most important primitive in this study. A square-activation strided CNN achieved the strongest high-accuracy true-FHE result, while a compact strided model achieved the lowest true-FHE la-

tency at lower accuracy. Future work should extend the FashionMNIST study to larger architectures and additional samples, test domain-specific private datasets such as medical images, and train with an explicit encrypted-cost proxy rather than relying on post-training compilation alone.

8. Contributions and Acknowledgements

Yavor Litchev is responsible for all contributions to this project, including problem formulation, literature review, dataset preparation, model implementation, training, Concrete ML compilation, simulated-FHE and true-FHE experiments, quantitative analysis, figure generation, report writing, code packaging, and poster preparation. The project is not shared with any other class, and there were no non-CS231N collaborators. No project-specific external starter code or public GitHub implementation was used as a base. Public software used in the project includes PyTorch, Concrete ML, and Brevitas, which are cited in the bibliography.

Generative AI usage statement. Generative AI assistance was not used in any way, shape, or form in this project. Yavor Litchev is responsible for all the code, written documents, experimental designs, and all other contributions. All numerical results reported in the paper come from the project experiments and associated artifacts, not from generated text.

References

- [1] A. Brutzkus, R. Gilad-Bachrach, and O. Elisha. Low latency privacy preserving inference. In *Proceedings of the 36th International Conference on Machine Learning*, 2019.
- [2] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachene. Tfhe: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33:34–91, 2020.
- [3] E. Chou, J. Beal, D. Levy, S. Yeung, A. Haque, and L. Fei-Fei. Faster cryptonets: Leveraging sparsity for real-world encrypted inference. *arXiv preprint arXiv:1811.09953*, 2018.
- [4] R. Dathathri, O. Saarikivi, H. Chen, K. Laine, K. Lauter, S. Maleki, M. Musuvathi, and T. Mytkowicz. Chet: An optimizing compiler for fully-homomorphic neural-network inferencing. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 142–156. ACM, 2019.
- [5] C. Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing*, pages 169–178. ACM, 2009.
- [6] R. Gilad-Bachrach, N. Dowlin, K. Laine, K. Lauter, M. Naehrig, and J. Wernsing. Cryptonets: Applying neural networks to encrypted data with high throughput and accuracy. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 201–210. PMLR, 2016.
- [7] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018.
- [8] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [10] H. Xiao, K. Rasul, and R. Vollgraf. Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.
- [11] Xilinx. Brevitas: Neural network quantization in pytorch. <https://github.com/Xilinx/brevitas>, 2026. Accessed May 2026.
- [12] Zama. Concrete ml: Privacy-preserving machine learning framework using fully homomorphic encryption. <https://github.com/zama-ai/concrete-ml>, 2026. Accessed May 2026.